

Resumen: Librerías de Python para Aprendizaje Automático - Caso Práctico

Contexto del Caso Práctico

Lorena, en prácticas en la empresa logística **Pick&Deliver**, debe desarrollar un modelo predictivo para cumplir plazos de entrega. Su mentor Miguel le ha retado a crear un modelo que prediga si se cumplirán los tiempos de entrega basándose en parámetros controlables por la empresa.

Librerías Clave para Machine Learning en Python

1. Pandas - Manipulación y Análisis de Datos

- **Creación:** 2008-2009, actualmente gestionado por NumFOCUS
- **Objeto principal:** DataFrame (estructura tabular versátil)
- **Funcionalidades principales:**
 - Carga de datos desde múltiples formatos (CSV, TSV, etc.)
 - Manipulación y transformación de datos
 - Análisis exploratorio (EDA)
 - Preparación para modelado

2. Matplotlib - Visualización de Datos

- Generación de gráficos y visualizaciones
- Integración con Pandas para análisis visual

3. Scikit-learn - Machine Learning Tradicional

- Algoritmos clásicos de ML
- Herramientas para preprocesamiento, modelado y evaluación

4. TensorFlow/Keras - Deep Learning

- Redes neuronales y aprendizaje profundo
- Keras como API de alto nivel sobre TensorFlow

5. PyTorch - Deep Learning

- Alternativa a TensorFlow
- Popular en investigación y desarrollo

Flujo de Trabajo Típico en Ciencia de Datos

1. **Carga de datos** → Pandas
2. **EDA (Análisis Exploratorio)** → Pandas + Matplotlib
3. **Preparación y tratamiento** → Pandas/Scikit-learn
4. **Generación del modelo** → Scikit-learn/TensorFlow/PyTorch
5. **Entrenamiento** → Librerías específicas
6. **Evaluación** → Scikit-learn

Ejemplos Prácticos con Pandas

Creación de DataFrames

```
# Desde array unidimensional
mydf1 = pd.DataFrame(np.array([1, 2, 3, 4, 5]))

# Desde array bidimensional con nombres personalizados
myarray = np.array([[10, 30, 20], [50, 40, 60]])
```

```
mydf = pd.DataFrame(myarray, index=['lentejas', 'espinacas'],  
                    columns=['enero', 'febrero', 'marzo'])
```

Operaciones Comunes

- **Acceso a columnas:** `mydf['enero']`
- **Dimensiones:** `mydf.shape` → (filas, columnas)
- **Estadísticas descriptivas:** `mydf.describe()`
- **Visualización básica:** `mydf.plot()`

Transformación de Datos

```
# Conversión categórico a numérico (binario)  
df['type'] = df['type'].map({'no_spam': 0, 'spam': 1})  
  
# Reemplazo de categorías múltiples  
shirts['size'] = shirts['size'].replace(regex='xlarge', value=4)
```

Filtrado y Selección

```
# Filtrar por condición  
shirts[shirts['size'].str.startswith('xl')]  
  
# Excluir valores  
shirts[shirts['size'] != 'xlarge']  
  
# Selección por posición  
shirts.iloc[2:5, 2] # Filas 3-5, columna 3
```

Visualización con Pandas/Matplotlib

```
# Gráfico básico  
mydf.plot()
```

```
# Trasposición y gráfico  
df = mydf.T  
df.plot()
```

Aplicación al Caso de Pick&Deliver

Lorena utilizaría principalmente **Pandas** para: 1. Cargar los datos históricos de entregas 2. Realizar EDA para entender las variables 3. Preparar los datos (limpieza, transformación) 4. Identificar patrones y correlaciones

Posteriormente emplearía **Scikit-learn** para: - Entrenar modelos predictivos (regresión, clasificación) - Evaluar el rendimiento del modelo - Optimizar hiperparámetros

Conclusión

Las librerías de Python para ML proporcionan un ecosistema completo para el desarrollo de soluciones predictivas. **Pandas** es fundamental en las etapas iniciales de manipulación y análisis de datos, mientras que librerías como **Scikit-learn**, **TensorFlow** y **PyTorch** se especializan en la creación y entrenamiento de modelos. El caso de Lorena ilustra cómo estas herramientas se aplican en problemas empresariales reales de logística y predicción.

Resumen de Librerías Esenciales para Ciencia de Datos en Python

Este documento presenta un recorrido práctico por cuatro librerías fundamentales en el ecosistema de ciencia de datos de Python: **Pandas**, **Matplotlib**, **Scikit-learn** y **TensorFlow/Keras**. A través de ejemplos de código y casos prácticos, se ilustra su uso para manipulación de datos, visualización, aprendizaje automático clásico y aprendizaje profundo.

1. Pandas: Manipulación y Análisis de Datos

Pandas es la librería estándar para trabajar con datos estructurados (tablas) en Python.

Operaciones Clave Demostradas:

- **Concatenación de DataFrames:** Se combinan dos DataFrames (`can_weather` y `us_weather`) que contienen datos meteorológicos de ciudades canadienses y estadounidenses usando `pd.concat()` .
- **Creación y Modificación de Columnas:**
 - Se añade una columna `'Total'` calculada como la suma de las filas (`axis=1`) de un DataFrame financiero (ingresos y costes).
 - Se crea una columna `'four'` como producto de otras columnas (`df['one'] * df['two']`).
 - Se modifica una columna existente (`'three'`) reasignándole nuevos valores.
- **Inserción y Eliminación de Columnas:**
 - `df.insert()` : Inserta una columna `'random'` con valores aleatorios en una posición específica (índice 1).
 - `df['flag'] = df['one'] > 2` : Crea una columna booleana basada en una condición.
 - `del df['two']` : Elimina una columna.
 - `df.pop('three')` : Extrae y elimina una columna, devolviendo su serie.
 - `df['foo'] = 'bar'` : Añade una columna con un valor constante para todas las filas.

Recurso: Documentación oficial y tutorial de 10 minutos para profundizar.

2. Matplotlib: Visualización de Datos

Matplotlib es la librería principal para crear visualizaciones estáticas, animadas e interactivas en Python.

Características y Uso:

- **Origen:** Creada para emular la capacidad gráfica de MATLAB, ahora es un estándar con una gran comunidad.

- **Submódulo principal:** `matplotlib.pyplot` (generalmente importado como `plt`).
- **Ejemplo de Gráfico de Dispersión (`scatter`):** `python plt.scatter('a', 'b', c='c', s='d', data=data) plt.xlabel('entry a') plt.ylabel('entry b') plt.show()`
 - `'a'` y `'b'` : Ejes X e Y.
 - `c='c'` : Colorea los puntos según la columna `'c'` .
 - `s='d'` : Define el tamaño de los puntos según la columna `'d'` .

Recurso: Se recomienda la documentación, el tutorial de inicio rápido y las "chuletas" (cheatsheets) para funciones comunes.

3. Scikit-learn: Aprendizaje Automático (Machine Learning)

Scikit-learn es la librería más popular para implementar algoritmos de aprendizaje automático clásico (supervisado y no supervisado).

Contexto y Aplicación:

- **Caso Práctico:** Predecir si un pedido llegará a tiempo (clasificación binaria) o predecir el tiempo de entrega (regresión).
- **Flujo de Trabajo Típico:**
 1. **Preparar datos:** Separar variables predictoras (`X`) de la variable objetivo (`y`). `python y = data['result'] X = data.drop('result', axis=1)`
 2. **Instanciar modelo:** Crear un objeto del algoritmo elegido (ej., `LogisticRegression()`).
 3. **Entrenar modelo:** Ajustar el modelo a los datos de entrenamiento con el método `.fit(X, y)` .
- **Ventaja:** Ofrece una API consistente para muchos algoritmos y herramientas auxiliares (preprocesamiento, validación, métricas).

Recurso: La documentación incluye un diagrama muy útil para elegir el algoritmo correcto según el tipo de problema.

4. TensorFlow y Keras: Aprendizaje Profundo (Deep Learning)

TensorFlow es un ecosistema completo para aprendizaje automático, y Keras es su API de alto nivel para construir y entrenar redes neuronales de forma sencilla.

Evolución e Importancia:

- **TensorFlow:** Liberado por Google en 2015, revolucionó el campo al hacer accesibles herramientas industriales de IA.
- **Keras:** Integrada en TensorFlow 2, simplifica enormemente la creación de modelos de deep learning con una interfaz orientada al usuario.

Ejemplo Práctico: Reconocimiento de Dígitos (MNIST)

1. **Carga de Datos:** El dataset MNIST (dígitos manuscritos) se carga directamente desde Keras.

```
python from keras.datasets import mnist (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```
2. **Construcción del Modelo:** Se usa el modelo `Sequential`, que permite apilar capas de neuronas (`Dense`).

```
python from keras import models, layers network = models.Sequential() network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,))) network.add(layers.Dense(10, activation='softmax'))
```

 - **Capa 1 (Dense):** 512 neuronas con función de activación `relu`.
 - **Capa 2 (Dense):** 10 neuronas (una por dígito) con activación `softmax` para obtener probabilidades.

Conclusión: Este conjunto de librerías forma un flujo de trabajo completo: **Pandas** prepara los datos, **Matplotlib** los explora visualmente, **Scikit-learn** permite crear modelos básicos rápidos, y **TensorFlow/Keras** posibilita el desarrollo de modelos complejos de aprendizaje profundo para problemas donde se requiere una mayor precisión.

Implementación de Redes Neuronales con Keras y PyTorch - Guía Comparativa

1. Arquitectura de Red Neuronal en Keras

Configuración del Modelo Secuencial

```
from keras import models
from keras import layers

network = models.Sequential()
network.add(layers.Dense(512, activation='relu', input_shape=(28*28,)))
network.add(layers.Dense(10, activation='softmax'))
```

Explicación: - **Capa 1:** 512 neuronas con activación ReLU (Rectified Linear Unit) - `input_shape=(28*28,)` : Recibe imágenes de 28x28 píxeles aplanadas a vectores de 784 elementos - Función ReLU: $f(x) = \max(0, x)$ - introduce no linealidad y evita el problema del gradiente desaparecido

- **Capa 2:** 10 neuronas con activación Softmax
- Produce distribución de probabilidad sobre 10 clases (dígitos 0-9)
- Softmax: convierte logits en probabilidades que suman 1

Configuración del Entrenamiento

```
network.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```


Parámetros clave: - **Optimizador RMSprop:** Adapta la tasa de aprendizaje para cada parámetro - **Pérdida:** Entropía cruzada categórica - ideal para clasificación multiclase - **Métrica:** Precisión (accuracy) - porcentaje de predicciones correctas

2. Preprocesamiento de Datos MNIST

Transformación de Imágenes

```
# Reestructuración y normalización
train_images = train_images.reshape((60000, 28*28))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28*28))
test_images = test_images.astype('float32') / 255
```

Propósito: - **Reshape:** Convierte imágenes 2D (28x28) a vectores 1D (784) - **Normalización:** Escala valores de píxeles de [0,255] a [0,1] - **Tipo float32:** Precisión adecuada para cálculos de red neuronal

Codificación de Etiquetas

```
from tensorflow.keras.utils import to_categorical
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

Ejemplo: Etiqueta "3" se convierte a [0,0,0,1,0,0,0,0,0]

3. Entrenamiento y Evaluación

Entrenamiento del Modelo

```
network.fit(train_images, train_labels, epochs=5, batch_size=128)
```

Resultados del entrenamiento: - Época 1: 92.53% precisión - Época 5: 98.85% precisión (conjunto de entrenamiento) - **Batch size 128:** Procesa 128 muestras antes de actualizar pesos - **5 epochs:** Pasa 5 veces por todo el dataset

Evaluación Final

```
test_loss, test_acc = network.evaluate(test_images, test_labels)
print("test_acc:", test_acc) # Resultado: 97.94%
```

Interpretación: El modelo generaliza bien con 97.94% de precisión en datos no vistos

4. Implementación Equivalente en PyTorch

Instalación e Importación

```
!pip install torch torchvision

import torch
import torch.nn as nn
import torchvision.datasets as dsets
import torchvision.transforms as transforms
from torch.autograd import Variable
```

Configuración de Parámetros

```
input_size = 784      # 28*28 píxeles
hidden_size = 500     # Neuronas en capa oculta
num_classes = 10      # Dígitos 0-9
num_epochs = 20       # Iteraciones completas
batch_size = 100      # Tamaño de lote
lr = 1e-3             # Tasa de aprendizaje
```

Definición de la Arquitectura

```
class Net(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

# Creación del modelo
net = Net(input_size, hidden_size, num_classes)
if torch.cuda.is_available():
    net.cuda() # Usar GPU si está disponible
```

Entrenamiento en PyTorch

```
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(net.parameters(), lr=lr)

for epoch in range(num_epochs):
    for images, labels in train_gen:
        images = Variable(images.view(-1, 28*28)).cuda()
        labels = Variable(labels).cuda()

        optimizer.zero_grad()
        outputs = net(images)
        loss = loss_function(outputs, labels)
        loss.backward()
        optimizer.step()
```

Diferencias clave con Keras: 1. **Definición explícita del forward pass** 2. **Cálculo manual de gradientes** (`loss.backward()`) 3. **Actualización manual de**

optimizador (optimizer.step()) 4. **Limpieza explícita de gradientes**
(optimizer.zero_grad())

Evaluación y Resultados

```
correct = 0
total = 0
for images, labels in test_gen:
    images = Variable(images.view(-1, 28*28)).cuda()
    labels = labels.cuda()
    output = net(images)
    _, predicted = torch.max(output, 1)
    correct += (predicted == labels).sum()
    total += labels.size(0)

print('Accuracy: %.3f %%' % ((100 * correct) / (total + 1)))
# Resultado: 98.28%
```

5. Comparación y Recomendaciones

Keras vs PyTorch

Aspecto	Keras	PyTorch
Sintaxis	Más simple, de alto nivel	Más explícita, flexible
Curva de aprendizaje	Suave	Más pronunciada
Depuración	Más abstracta	Más transparente
Comunidad	Amplia en producción	Fuerte en investigación
Precisión MNIST	~97.94%	~98.28%

Recursos Recomendados

Para Keras: - Documentación oficial: <https://keras.io/api/> - Repositorio de ejemplos: <https://keras.io/examples/> - Datasets integrados: <https://keras.io/api/datasets/>

Para PyTorch: - Documentación API: <https://pytorch.org/docs/stable/index.html> - Modelos pre-entrenados: <https://pytorch.org/hub/> - Tutoriales oficiales: <https://pytorch.org/tutorials/> - MOOC fast.ai: <https://course.fast.ai/>

Caso Práctico: Lorena

Lorena descubrió que PyTorch ofrece mayor precisión (~98.28% vs ~97.94%) pero requiere más especialización. La decisión depende del equilibrio entre: 1. **Rapidez de desarrollo** → Keras 2. **Control y optimización** → PyTorch 3. **Requisitos de precisión** → Evaluar ambos

Conclusión: Ambas librerías son excelentes. Keras es ideal para prototipado rápido, mientras que PyTorch ofrece mayor control para investigación y optimización avanzada.